NDP2

30 Mart 2026 Pazartesi 10:09

KALITIM:

Kalıtım'ın en önemli özelliği sınıfı dış dünyaya açmasıdır. (Yeni bir sınıf türetileceğinde eski özellikleri kolayca yeni sınıfta kullanmak)
2.önemli özelliği tanımı genişletebilmesidir. (kod tekrarını önler)

Open/Closed principle bir sınıf genişletilmeye açık değiştirilmeye kapalı olmalıdır. (yeni özellik eklenirken)

oop için bir dilin kalıtım ve soyut sınıf içermesi zorunluluktur. Interface ise opsiyoneldir.
Arayüz aslında her metodu soyut olan bir soyut sınıftır.

Java ve C# dilinde arayüz bulunurken C++'da bulunmaz.

INTERFACE:

- Arayüz kullanımı aslında bir sözleşme imzalamaktır.
- Bir arayüzden kalıtım alan bir sınıf o arayüzü gerçekleştireceğini vadediyor demektir.
- Arayüzler yardımıyla kullanılabilirlik dış dünyaya rahatlıkla açılabilir.
- Java ve C#'ta arayüzler alan içeremez sadece public metot tanımı içerebilirler.

```
interface IEmail{
    public void setGonderen(String gonderen);
    public void setAlan(String alan);
    public void setMesaj(IIcerik mesaj);
}
interface IIcerik{
    public String StringOlarakIcerik();
}
class Email implements IEmail{
    public void setGonderen(String gonderen){ /*Göndereni ayarlar*/ }
    public void setAlan(String alan){ /*Alanı ayarlar*/ }
    public void setMesaj(IIcerik mesaj){ /*Mail mesajını ayarlar*/ }
}
```

```
class Html implements IIcerik{
    public String StringOlarakIcerik(){
        // HTML'i String olarak ifade et.
    }
}
```

C++:	C#	Java:
Çoklu sınıf kalıtımı	Çoklu Interface kalıtımı	Çoklu Interface kalıtımı
sınıfın ismiyle	base	super

```
class Sekil
{
    protected double genislik;
    protected double yukseklik;
    1 başvuru
    public Sekil(double genislik,double yukseklik)
    {
        this.genislik = genislik;
        this.yukseklik = yukseklik;
    }
}
```

C# base

```
class Daire : Sekil
{
    0 başvuru
    public Daire(double g,double y) : base(g,y)
    {
    }
}
```

C++

```
class Insan{
    protected:
        double boy;
    public:
        Insan(double boy) { this->boy = boy; }
};
class Calisan{
    protected:
        int Id;
    public:
        Calisan(int Id){ this->Id = Id; }
};
class Ogretmen : public Insan, public Calisan{
    public:
        Ogretmen(double boy,int Id) : Insan(boy),Calisan(Id){
        }
};
```

Python

```
class Insan:
    ...
class Calisan:
    ...
class Ogretmen(Insan, Calisan):
    super()
```

Yazılan sıraya göre çağırır
Önce Insan sonra Calisan sınıfın Kurucusu çağrılır.

Namespace ve Paketler:
C++ ve C#ta aynı isimdeki sınıf ve metodları ayırmak için namespace kullanılır.
Javada bunun için paketler vardır. Paketler klasörleme mantığıyla ayırırlar.

```
namespace A{
    double Hesapla() { return 0; }
}
namespace B{
    double Hesapla() { return 1; }
}
int main(){
    cout<<A::Hesapla()<<endl; 0 yazar
    cout<<B::Hesapla()<<endl; 1 yazar
    return 0;
}
```
C++

Soyutlama Kavramı:
Karmaşık sistemlerin gereksiz detaylarından arındırılarak sadece temel özelliklerinin sunulmasıdır.

- sistemin nasıl çalıştığı detayları gizlenir.
- sistemin ne olduğu bilgisi kullanıcı ve diğer sınıflara sunulur.

Soyut Sınıflar:

C++

```
class Canli{
    private:
        int yas;
    public:
        virtual void YemekYe() = 0;
        int Yas()const { return yas; }
};
class Kedi : public Canli{
    public:
        void YemekYe(){

        }
};
```

C#

```
public abstract class Canli{

    public int yas { get; }

    public abstract void YemekYe();
}

public class Kedi : Canli
{

    public override void YemekYe()
    {

    }
}
```

Kalıtım Hiyerarşisi (C# ve Java):

Üst Yapı	Arayüz	Arayüz	Sınıf	Soyut Sınıf	Sınıf	Arayüz	Soyut Sınıf
Alt Yapı	Arayüz	Sınıf	Sınıf	Sınıf	Soyut Sınıf	Soyut Sınıf	Soyut Sınıf

Object Veri Türü

- Herhangi bir türü içinde barındırabilecek şekilde tasarlanmış veri türüdür.
- Boxing ve Unboxing kullanımlarında büyük önem arz eder.
- Java dilinde en üstteki sınıf Object sınıfıdır ve bu sınıftan diğer bütün sınıflar gizli olarak kalıtım alır.
- Java ve C# dillerinde Object sınıfı bulunurken, C++ dilinde böyle bir sınıf yoktur.

Generic – Şablon Yapılar

- C# ve C++ dillerinde aktif olarak kullanılan şablon yapılar Java diline sonradan dahil olmuştur.
- Bir dilde Object sınıfı ile şablon yapıları taklit edilebilir.

C#

```
public class Koleksiyon<Tur>
{
    public Tur[] Elemanlar;

    public Koleksiyon(Tur []Elemanlar)
    {
        this.Elemanlar = Elemanlar;
    }

    public String EnBuyukEnKucukBirlestir()
    {
        return Elemanlar.Max().ToString() + Elemanlar.Min().ToString();
    }
```

```
int[] tamsayilar = { 15,95,20,2,18,32};
Koleksiyon<int> koleksiyonsayilar = new Koleksiyon<int>(tamsayilar);
koleksiyonsayilar.EnBuyukEnKucukBirlestir();   952 döner

char[] karakterler = { 'a', 'w', 'r', 't', 'k', 'p' };
Koleksiyon<char> koleksiyonkarakterler = new Koleksiyon<char>(karakterler);
koleksiyonkarakterler.EnBuyukEnKucukBirlestir();   wa döner
```

C++

```
template <typename Tur>
class Koleksiyon{
    public:
        Tur *Elemanlar;
        int uzunluk;
        Koleksiyon(Tur *Elemanlar,int uzunluk){
            this->Elemanlar = Elemanlar;
            this->uzunluk = uzunluk;
        }
        string EnBuyukEnKucukBirlestir(){
            return toString(*max_element(Elemanlar,Elemanlar+uzunluk))+
                   toString(*min_element(Elemanlar,Elemanlar+uzunluk));
        }
        string toString(Tur t)
        {
            ostringstream ss;
            ss << t;
            return ss.str();
        }

};
```

friend: bir sınıfa veya metoda friend keyword eklendiğinde artık o sınıf veya metod friend yazıldığı sınıfın private alanlarına erişim sağlayabilir. Javada yoktur.

Java 8'den sonra **default fonksiyon** olarak interface içine gövdesi olan metodlar

eklenebilir. Yıllar önce yazılmış ve birçok sınıftan kullanılanarayüze yeni metot eklendiğinde mevcut sınıflarda implement etmemek için geliştirilmiştir. Kısa ve temel bir gövde olmalıdır.